

Formatting Ranges

Document #: P2286R3
Date: 2021-11-16
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Implementation Experience](#)

[3 Proposal Considerations](#)

[3.1 What types to print?](#)

[3.2 What representation?](#)

[3.2.1 vector \(and other ranges\)](#)

[3.2.2 pair and tuple](#)

[3.2.3 map and set \(and other associative containers\)](#)

[3.2.4 char and string \(and other string-like types\) in ranges or tuples](#)

[3.2.5 Format Specifiers](#)

[3.2.6 Explanation of Added Specifiers](#)

[3.3 Implementation Challenges](#)

[3.3.1 Wrapping basic format context is not generally possible](#)

[3.3.2 Manipulating basic format parse context to search for sentinels](#)

[3.3.3 Parsing of alignment, padding, and width](#)

[3.4 How to support those views which are not const-iterable?](#)

[3.5 What additional functionality?](#)

[3.6 format or std::cout?](#)

[3.7 What about vector<bool>?](#)

[4 Proposal](#)

[4.1 Wording](#)

[5 References](#)

§ 1 Revision History

Since [[P2286R2](#)], several major changes:

- This paper assumes the adoption of [[P2418R0](#)], which affects how [non-const-iterable views](#) are handled. This paper now introduces two concepts (`formattable` and `const_formattable`) instead of just one.
- Extended discussion and functionality for various [representations](#), including how to quote strings properly and how to format associative ranges.
- Introduction of format specifiers of all kinds and discussion of how to make them work more broadly.
- Removed the wording, since the priority is the design.

Since [\[P2286R1\]](#), adding a sketch of wording.

[\[P2286R0\]](#) suggested making all the formatting implementation-defined. Several people reached out to me suggesting in no uncertain terms that this is unacceptable. This revision lays out options for such formatting.

§ 2 Introduction

[\[LWG3478\]](#) addresses the issue of what happens when you split a string and the last character in the string is the delimiter that you are splitting on. One of the things I wanted to look at in research in that issue is: what do *other* languages do here?

For most languages, this is a pretty easy proposition. Do the split, print the results. This is usually only a few lines of code.

Python:

```
print("xyx".split("x"))
```

outputs

```
['', 'y', '']
```

Java (where the obvious thing prints something useless, but there's a non-obvious thing that is useful):

```
import java.util.Arrays;

class Main {
    public static void main(String args[]) {
        System.out.println("xyx".split("x"));
        System.out.println(Arrays.toString("xyx".split("x")));
    }
}
```

outputs

```
[Ljava.lang.String;@76ed5528
[, y]
```

Rust (a couple options, including also another false friend):

```
use itertools::Itertools;

fn main() {
    println!("{:?}", "xyx".split('x'));
    println!("[{}]", "xyx".split('x').format(", "));
    println!("{:?}", "xyx".split('x').collect::<Vec<_>>());
}
```

outputs

```
Split(SplitInternal { start: 0, end: 3, matcher: CharSearcher { haystack: "xyx",
    finger: 0, finger_back: 3, needle: 'x', utf8_size: 1, utf8_encoded: [120,
    0, 0, 0] }, allow_trailing_empty: true, finished: false })
[, y, ]
["", "y", ""]
```

Kotlin:

```
fun main() {  
    println("xyx".split("x"));  
}
```

outputs

```
[, y, ]
```

Go:

```
package main  
import "fmt"  
import "strings"  
  
func main() {  
    fmt.Println(strings.Split("xyx", "x"));  
}
```

outputs

```
[ y ]
```

JavaScript:

```
console.log('xyx'.split('x'))
```

outputs

```
[ '', 'y', '' ]
```

And so on and so forth. What we see across these languages is that printing the result of split is pretty easy. In most cases, whatever the print mechanism is just works and does something meaningful. In other cases, printing gave me something other than what I wanted but some other easy, provided mechanism for doing so.

Now let's consider C++.

```
#include <iostream>  
#include <string>  
#include <ranges>  
#include <format>  
  
int main() {  
    // need to predeclare this because we can't split an rvalue string  
    std::string s = "xyx";  
    auto parts = s | std::views::split('x');  
  
    // nope  
    std::cout << parts;  
  
    // nope (assuming std::print from P2093)  
    std::print("{} ", parts);  
  
    std::cout << "[";  
    char const* delim = "";  
    for (auto part : parts) {
```

```

std::cout << delim;

// still nope
std::cout << part;

// also nope
std::print("{} ", part);

// this finally works
std::ranges::copy(part, std::ostream_iterator<char>(std::cout));

// as does this
for (char c : part) {
    std::cout << c;
}
delim = ", ";
}
std::cout << "]\n";
}

```

This took me more time to write than any of the solutions in any of the other languages. Including the Go solution, which contains 100% of all the lines of Go I've written in my life.

Printing is a fairly fundamental and universal mechanism to see what's going on in your program. In the context of ranges, it's probably the most useful way to see and understand what the various range adapters actually do. But none of these things provides an `operator<<` (for `std::cout`) or a formatter specialization (for `format`). And the further problem is that as a user, I can't even do anything about this. I can't just provide an `operator<<` in `namespace std` or a very broad specialization of `formatter` - none of these are program-defined types, so it's just asking for clashes once you start dealing with bigger programs.

The only mechanisms I have at my disposal to print something like this is either

1. nested loops with hand-written delimiter handling (which are tedious and a bad solution), or
2. at least replace the inner-most loop with a `ranges::copy` into an output iterator (which is more differently bad), or
3. Write my own formatting library that I *am* allowed to specialize (which is not only bad but also ridiculous)
4. Use `fmt::format`.

§ 2.1 Implementation Experience

That's right, there's a fourth option for C++ that I haven't shown yet, and that's this:

```

#include <ranges>
#include <string>
#include <fmt/ranges.h>

int main() {
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    fmt::print("{}\n", parts);
    fmt::print("[{}]\n", fmt::join(parts, ","));
}

```

outputting

```
{{}, {'y'}}  
[{}, {'y'}]
```

And this is great! It's a single, easy line of code to just print arbitrary ranges (include ranges of ranges).

And, if I want to do something more involved, there's also `fmt::join`, which lets me specify both a format specifier and a delimiter. For instance:

```
std::vector<uint8_t> mac = {0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};  
fmt::print("{:02x}\n", fmt::join(mac, ":"));
```

outputs

```
aa:bb:cc:dd:ee:ff
```

`fmt::format` (and `fmt::print`) solves my problem completely. `std::format` does not, and it should.

§ 3 Proposal Considerations

The Ranges Plan for C++23 [P2214R0] listed as one of its top priorities for C++23 as the ability to format all views. Let's go through the issues we need to address in order to get this functionality.

§ 3.1 What types to print?

The standard library is the only library that can provide formatting support for standard library types and other broad classes of types like ranges. In addition to ranges (both the concrete containers like `vector<T>` and the range adaptors like `views::split`), there are several very commonly used types that are currently not printable.

The most common and important such types are `pair` and `tuple` (which ties back into Ranges even more closely once we adopt `views::zip` and `views::enumerate`). `fmt` currently supports printing such types as well:

```
fmt::print("{}\n", std::pair(1, 2));
```

outputs

```
(1, 2)
```

Another common and important set of types are `std::optional<T>` and `std::variant<Ts...>`. `fmt` does not support printing any of the sum types. There is not an obvious representation for them in C++ as there might be in other languages (e.g. in Rust, an `Option<i32>` prints as either `Some(42)` or `None`, which is also the same syntax used to construct them).

However, the point here isn't necessarily to produce the best possible representation (users who have very specific formatting needs will need to write custom code anyway), but rather to provide something useful. And it'd be useful to print these types as well. However, given that `optional` and `variant` are both less closely related to Ranges than `pair` and `tuple` and also have less obvious representation, they are less important.

§ 3.2 What representation?

There are several questions to ask about what the representation should be for printing. I'll go through each kind in turn.

§ 3.2.1 vector (and other ranges)

Should `std::vector<int>{1, 2, 3}` be printed as `{1, 2, 3}` or `[1, 2, 3]`? At the time of [P2286R1], `fmt` used `{}`s but changed to use `[]`s for consistency with Python (400b953f).

Even though in C++ we initialize vectors (and, generally, other containers as well) with `{}`s while Python's uses `[1, 2, 3]` (and likewise Rust has `vec![1, 2, 3]`), `[]` is typical representationally so seems like the clear best choice here.

§ 3.2.2 pair and tuple

Should `std::pair<int, int>{4, 5}` be printed as `{4, 5}` or `(4, 5)`? Here, either syntax can claim to be the syntax used to initialize the pair/tuple. `fmt` has always printed these types with `()`s, and this is also how Python and Rust print such types. As with using `[]` for ranges, `()` seems like the common representation for tuples and so seems like the clear best choice.

§ 3.2.3 map and set (and other associative containers)

Should `std::map<int, int>{{1, 2}, {3, 4}}` be printed as `[(1, 2), (3, 4)]` (as follows directly from the two previous choices) or as `{1: 2, 3: 4}` (which makes the *association* clearer in the printing)? Both Python and Rust print their associating containers this latter way.

The same question holds for sets as well as maps, it's just a question for whether `std::set<int>{1, 2, 3}` prints as `[1, 2, 3]` (i.e. as any other range of `int`) or `{1, 2, 3}`?

If we print maps as any other range of pairs, there's nothing left to do. If we print maps as associations, then we additionally have to answer the question of how user-defined associative containers can get printed in the same way. Hold onto this thought for a minute.

§ 3.2.4 char and string (and other string-like types) in ranges or tuples

Should `pair<char, string>('x', "hello")` print as `(x, hello)` or `('x', "hello")`? Should `pair<char, string>('y', "with\n\"quotes\")` print as:

```
(y, with
"quotes")
```

or

```
('y', "with\n\"quotes\"")
```

While `char` and `string` are typically printed unquoted, it is quite common to print them quoted when contained in tuples and ranges (as Python, Rust, and `fmt` do). Rust escapes internal strings, so prints as `('y', "with\n\"quotes\"")` (the Rust implementation of `Debug` for `str` can be found [here](#) which is implemented in terms of [escape_debug_ext](#)). Following discussion of this paper and this design, Victor Zverovich implemented in this `fmt` as well.

Escaping seems like the most desirable behavior. Following Rust's behavior, we escape `\t`, `\r`, `\n`, `\\`, `"` (for string types only), `'` (for `char` types only), and extended graphemes (if Unicode).

Also, `std::string` isn't the only string-like type: if we decide to print strings quoted, how do users opt in to this behavior for their own string-like types? And `char` and `string` aren't the only types that may desire to have some kind of *debug* format and some kind of regular format, how to differentiate those?

Moreover, it's all well and good to have the default formatting option for a range or tuple of strings to be printing those strings escaped. But what if users want to print a range of strings *unescaped*? I'll get back to this.

§ 3.2.5 Format Specifiers

One of (but hardly the only) the great selling points of `format` over `iostreams` is the ability to use specifiers. For instance, from the `fmt` documentation:

```
fmt::format("{:<30}", "left aligned");  
// Result: "left aligned"  
fmt::format("{:>30}", "right aligned");  
// Result: "right aligned"  
fmt::format("{:^30}", "centered");  
// Result: "centered"  
fmt::format("{:*^30}", "centered"); // use '*' as a fill char  
// Result: "*****centered*****"
```

Earlier revisions of this paper suggested that formatting ranges and tuples would accept no format specifiers, but there indeed are quite a few things we may want to do here (as by Tomasz Kamiński and Peter Dimov):

- Formatting a range of pairs as a map (the `key: value` syntax rather than the `(key, value)` one)
- Formatting a range of chars as a string (i.e. to print `hello` or `"hello"` rather than `['h', 'e', 'l', 'l', 'o']`)

But these are just providing a specifier for how we format the range itself. How about how we format the elements of the range? Can I conveniently format a range of integers, printing their values as hex? Or as characters? Or print a range of chrono time points in whatever format I want? That's fairly powerful.

The problem is how do we actually *do that*. After a lengthy discussion with Peter Dimov, Tim Song, and Victor Zverovich, this is what we came up with. I'll start with a table of examples and follow up with a more detailed explanation.

Instead of writing a bunch of examples like `print("{:??}\n", v)`, I'm just displaying the format string in one column (the `"{:??}"` here) and the argument in another (the `v`):

Format String	Contents	Formatted Output
<code>{}</code>	<code>"hello"s</code>	<code>hello</code>
<code>{:??}</code>	<code>"hello"s</code>	<code>"hello"</code>
<code>{}</code>	<code>vector{"hello"s, "world"s}</code>	<code>["hello", "world"]</code>
<code>{:}</code>	<code>vector{"hello"s, "world"s}</code>	<code>["hello", "world"]</code>
<code>{:??}</code>	<code>vector{"hello"s, "world"s}</code>	<code>["hello", "world"]</code>
<code>{:*^14}</code>	<code>vector{"he"s, "wo"s}</code>	<code>*["he", "wo"]*</code>
<code>{:.*^14}</code>	<code>vector{"he"s, "wo"s}</code>	<code>[*****he*****, *****wo*****]</code>
<code>{:}</code>	<code>42</code>	<code>42</code>
<code>{:#x}</code>	<code>42</code>	<code>0x2a</code>

Format String	Contents	Formatted Output
{}	vector<char>{'H', 'e', 'l', 'l', 'o'}	['H', 'e', 'l', 'l', 'o']
{::}	vector<char>{'H', 'e', 'l', 'l', 'o'}	[H, e, l, l, o]
{::?c}	vector<char>{'H', 'e', 'l', 'l', 'o'}	['H', 'e', 'l', 'l', 'o']
{::d}	vector<char>{'H', 'e', 'l', 'l', 'o'}	[72, 101, 108, 108, 111]
{::#x}	vector<char>{'H', 'e', 'l', 'l', 'o'}	[0x48, 0x65, 0x6c, 0x6c, 0x6f]
{:s}	vector<char>{'H', 'e', 'l', 'l', 'o'}	Hello
{:?s}	vector<char>{'H', 'e', 'l', 'l', 'o'}	"Hello"
{}	pair{42, "hello"s}	(42, "hello")
{::#x:^10}	pair{42, "hello"s}	(0x2a, **hello**)
{: #x ^10}	pair{42, "hello"s}	(0x2a, **hello**)
{}	vector{pair{42, "hello"s}}	[(42, "hello")]
{:m}	vector{pair{42, "hello"s}}	{42: "hello"}
{:m::#x:^10}	vector{pair{42, "hello"s}}	{0x2a: **hello**}
{}	vector<vector{'a'}, vector{'b', 'c'}>	[['a'], ['b', 'c']]
{::?s}	vector{vector{'a'}, vector{'b', 'c'}}}	["a", "bc"]
{:::d}	vector{vector{'a'}, vector{'b', 'c'}}}	[[97], [98, 99]]
{}	pair(system_clock::now(), system_clock::now())	(2021-10-24 20:33:37, 2021-10-24 20:33:37)
{: %Y-%m-%d %H:%M:%S}	pair(system_clock::now(), system_clock::now())	(2021-10-24, 20:33:37)

§ 3.2.6 Explanation of Added Specifiers

§ 3.2.6.1 The debug specifier ?

`char` and `string` and `string_view` will start to support the `?` specifier. This will cause the character/string to be printed as quoted (characters with `'` and strings with `"`) and all characters to be escaped, as [described earlier](#).

This facility will be generated by the formatters for these types providing an addition member function (on top of `parse` and `format`):

```
void format_as_debug();
```

Which other formatting types may conditionally invoke when they parse a `?`. For instance, since the intent is that range formatters print escaped by default, the logic for a simple range formatter that accepts no specifiers might look like this (note that this paper is proposing something more complicated than this, this is just an example):


```

template <typename V>
struct range_formatter {
    std::formatter<V> underlying;

    template <typename ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        // ensure that the format specifier is empty
        if (ctx.begin() != ctx.end() && *ctx.begin() != '}') {
            throw std::format_error("invalid format");
        }

        // ensure that the underlying type can parse an empty specifier
        auto out = underlying.parse(ctx);

        // conditionally format as debug, if the type supports it
        if constexpr (requires { underlying.format_as_debug(); }) {
            underlying.format_as_debug();
        }
        return out;
    }
};

template <typename R, typename FormatContext>
requires
    std::same_as<std::remove_cvref_t<std::ranges::range_reference_t<R>>, V>
constexpr auto format(R&& r, FormatContext& ctx) {
    auto out = ctx.out();
    *out++ = '[';
    auto first = std::ranges::begin(r);
    auto last = std::ranges::end(r);
    if (first != last) {
        // have to format every element via the underlying formatter
        ctx.advance_to(std::move(out));
        out = underlying.format(*first, ctx);
        for (++first; first != last; ++first) {
            *out++ = ',';
            *out++ = ' ';
            ctx.advance_to(std::move(out));
            out = underlying.format(*first, ctx);
        }
    }
    *out++ = ']';
    return out;
};

```

§ 3.2.6.2 Range specifiers

Range format specifiers come in two kinds: specifiers for the range itself and specifiers for the underlying elements of the range. They must be provided in order: the range specifiers (optionally), then if desired, a colon and then the underlying specifier (optionally). For instance:

specifier	meaning
{}	No specifiers
{:}	No specifiers
{:<10}	The whole range formatting is left-aligned, with a width of 10

specifier	meaning
<code>{: *^20}</code>	The whole range formatting is center-aligned, with a width of 20, padded with <code>*</code> s
<code>{:m}</code>	Apply the <code>m</code> specifier to the range
<code>{::d}</code>	Apply the <code>d</code> specifier to each element of the range
<code>{: ?s}</code>	Apply the <code>?s</code> specifier to the range
<code>{:m::#x:#x}</code>	Apply the <code>m</code> specifier to the range and the <code>:#x:#x</code> specifier to each element of the range

There are only a few top-level range-specific specifiers proposed:

- `s`: for ranges of char, only: formats the range as a string.
- `?s` for ranges of char, only: same as `s` except will additionally quote and escape the string
- `m`: for ranges of pairs (or tuples of size 2) will format as `{k1: v1, k2: v2}` instead of `[(k1, v1), (k2, v2)]` (i.e. as a map).
- `e`: will format without the `[]`s. This will let you, for instance, format a range as `a, b, c` or `{a, b, c}` or `(a, b, c)` or however else you want, simply by providing the desired format string.

Additionally, ranges will support the same fill/align/width specifiers as in *std-format-spec*, for convenience and consistency.

If no element-specific formatter is provided (i.e. there is no inner colon - an empty element-specific formatter is still an element-specific formatter), the range will be formatted as debug. Otherwise, the element-specific formatter will be parsed and used.

To revisit a few rows from the earlier table:

Format String	Contents	Formatted Output
<code>{}</code>	<code>vector<char>{'H', 'e', 'l', 'l', 'o'}</code>	<code>['H', 'e', 'l', 'l', 'o']</code>
<code>{::}</code>	<code>vector<char>{'H', 'e', 'l', 'l', 'o'}</code>	<code>[H, e, l, l, o]</code>
<code>{::?c}</code>	<code>vector<char>{'H', 'e', 'l', 'l', 'o'}</code>	<code>['H', 'e', 'l', 'l', 'o']</code>
<code>{::d}</code>	<code>vector<char>{'H', 'e', 'l', 'l', 'o'}</code>	<code>[72, 101, 108, 108, 111]</code>
<code>{::#x}</code>	<code>vector<char>{'H', 'e', 'l', 'l', 'o'}</code>	<code>[0x48, 0x65, 0x6c, 0x6c, 0x6f]</code>
<code>{:s}</code>	<code>vector<char>{'H', '\t', 'l', 'l', 'o'}</code>	<code>H llo</code>
<code>{: ?s}</code>	<code>vector<char>{'H', '\t', 'l', 'l', 'o'}</code>	<code>"H\tllo"</code>
<code>{}</code>	<code>vector{vector{'a'}, vector{'b', 'c'}}</code>	<code>[['a'], ['b', 'c']]</code>

Format String	Contents	Formatted Output
{::?s}	vector{vector{'a'}, vector{'b', 'c'}}	["a", "bc"]
{:::d}	vector{vector{'a'}, vector{'b', 'c'}}	[[97], [98, 99]]

The second row is not printed quoted, because an empty element specifier is provided. The third row is printed quoted again because it was explicitly asked for using the `?c` specifier, applied to each character.

The last row, `:::d`, is parsed as:

	top level outer vector		top level inner vector		inner vector each element
:	(none)	:	(none)	:	d

That is, the `d` format specifier is applied to each underlying `char`, which causes them to be printed as integers instead of characters.

Note that you can provide both a fill/align/width specifier to the range itself as well as to each element:

Format String	Contents	Formatted Output
{}	vector<int>{1, 2, 3}	[1, 2, 3]
{::*^5}	vector<int>{1, 2, 3}	[**1**, **2**, **3**]
{:o^17}	vector<int>{1, 2, 3}	0000[1, 2, 3]0000
{:o^29:*^5}	vector<int>{1, 2, 3}	0000[**1**, **2**, **3**]0000

§ 3.2.6.3 Pair and Tuple Specifiers

This is the hard part.

To start with, we for consistency will support the same fill/align/width specifiers as usual.

But for ranges, we can have the underlying element's formatter simply parse the whole format specifier string from the character past the `:` to the `}`. The range doesn't care anymore at that point, and what we're left with is a specifier that the underlying element should understand (or not).

For `pair`, it's not so easy, because format strings can contain *anything*. Absolutely anything. So when trying to parse a format specifier for a `pair<X, Y>`, how do you know where `X`'s format specifier ends and `Y`'s format specifier begins? This is, in general, impossible.

But Tim's insight was to take a page out of `sed`'s book and rely on the user providing the specifier string to actually know what they're doing, and thus provide their own delimiter. `pair` will recognize the first character that is not one of its formatters as the delimiter, and then delimit based on that.

Let's start with some easy examples:

Format String	Contents	Formatted Output
{}	pair(10, 1729)	(10, 1729)
{:}	pair(10, 1729)	(10, 1729)

Format String	Contents	Formatted Output
{::#x:04X}	pair(10, 1729)	(0xa, 06C1)
{: #x 04X}	pair(10, 1729)	(0xa, 06C1)
{:Y#xY04X}	pair(10, 1729)	(0xa, 06C1)

In the first two rows, there are no specifiers for the underlying elements. The last three rows each provide the same specifiers, but use a different delimiter:

	pair specifier	delimiter	first specifier	delimiter	second specifier
:	(none)	:	#x	:	04X
:	(none)		#x		04X
:	(none)	Y	#x	Y	04X

If you provide the first specifier, you must provide all the specifiers. In other words, `:#x` would be an invalid format specifier for a `pair<int, int>`.

To demonstrate why such a scheme is necessary, and simply using `:` as a delimiter is insufficient, consider chrono formatters. Chrono format strings allow anything, including `:`. Consider trying to format `std::chrono::system_clock::now()` using various specifiers:

Format String	Formatted Output
{}	2021-10-24 20:33:37
{:%Y-%m-%d}	2021-10-24
{:%H:%M:%S}	20:33:37
{:%H hours, %M minutes, %S seconds}	20 hours, 33 minutes, 37 seconds

How could `pair` possibly know when to stop parsing first's specifier given... that? It can't. But if allow an arbitrary choice of delimiter, the user can pick one that won't interfere:

Format String	Contents	Formatted Output
{}	pair(now(), 1729)	(2021-10-24 20:33:37, 1729)
{:m :%Y-%m-%d #x}	pair(now(), 1729)	2021-10-24: 0x6c1

Which is parsed as:

	pair specifier	delimiter	first specifier	delimiter	second specifier
:	m		:%Y-%m-%d		#x

The above also introduces the only top-level specifier for `pair`: `m`. As with Ranges described in the previous section (and, indeed, necessary to support the Ranges functionality described there), the `m` specifier formatters pairs and 2-tuples as associations (i.e. `k: v`) instead of as a pair/tuple (i.e. `(k, v)`):

Format String	Contents	Formatted Output
{}	pair(1, 2)	(1, 2)
{:m}	pair(1, 2)	1: 2
{:m}	tuple(1, 2)	1: 2
{}	tuple(1)	(1)

Format String	Contents	Formatted Output
<code>{:m}</code>	<code>tuple(1)</code>	ill-formed
<code>{}</code>	<code>tuple(1,2,3)</code>	(1, 2, 3)
<code>{:m}</code>	<code>tuple(1,2,3)</code>	ill-formed

Similarly to how in the debug specifier is handled by introducing a:

```
void format_as_debug();
```

function, pair and tuple will provide a:

```
void format_as_map();
```

function, that for tuple of size other than 2 will throw an exception (since you cannot format those as a map).

§ 3.3 Implementation Challenges

I implemented the range and pair/tuple portions of this proposal on top of libfmt. I chose to do it on top so that I can easily [share the implementation](#), as such I could not implement `?` support for strings and char, though that is not a very interesting part of this proposal (at least as far as implementability is concerned). There were two big issues that I ran into that are worth covering.

§ 3.3.1 Wrapping `basic_format_context` is not generally possible

In order to be able to provide an arbitrary type's specifiers to format a range, you have to have a `formatter<V>` for the underlying type and use that specific formatter in order to parse the format specifier and then format into the given context. If that's all you're doing, this isn't that big a deal, and I showed a simplified implementation of `range_formatter<V>` [earlier](#).

However, if you additionally want to support fill/pad/align, then the game changes. You can't format into the provided context - you have to format into *something else* first and then do the adjustments later. Adding padding support ends up doing something more like this:

No padding	With padding
<pre>template <typename R, typename FormatContext> constexpr auto format(R&& r, FormatContext& ctx) { auto out = ctx.out(); *out++ = '['; auto first = std::ranges::begin(r); auto last = std::ranges::end(r); if (first != last) { ctx.advance_to(std::move(out)); out = underlying.format(*first, ctx); for (++first; first != last; ++first) { *out++ = ','; *out++ = ' '; } ctx.advance_to(std::move(out)); } }</pre>	<pre>template <typename R, typename FormatContext> constexpr auto format(R&& r, FormatContext& ctx) { fmt::memory_buffer buf; fmt::basic_format_context<fmt::appender, char> bctx(appender(buf), ctx.args(), ctx.locale()); auto out = bctx.out(); *out++ = '['; auto first = std::ranges::begin(r); auto last = std::ranges::end(r); if (first != last) { bctx.advance_to(std::move(out)); out = underlying.format(*first, bctx); for (++first; first != last; ++first) { *out++ = ','; *out++ = ' '; } } }</pre>

No padding	With padding
<pre> out = underlying.format(*first, ctx); } } *out++ = ']'; return out; } </pre>	<pre> bctx.advance_to(std::move(out)); out = underlying.format(*first, bctx); } } *out++ = ']'; return fmt::write(ctx.out(), fmt::string_view(buf.data(), buf.size()), this->specs); } </pre>

It's mostly the same - we format into `bctx` instead of `ctx` and then write into `ctx` later using the specs that we already parsed. The code seems straightforward enough, except...

First, we don't even expose a way to construct `basic_format_context` so can't do this at all. Nor do we expose a way of constructing an iterator type for formatting into some buffer. And if we could construct these things, the real problem hits when we try to construct this new context. We need some kind of

`fmt::basic_format_context<???, char>`, and we need to write into some kind of dynamic buffer, so

`fmt::appender` is the appropriate choice for iterator. But the issue here is that

`fmt::basic_format_context<Out, CharT>` has a member

`fmt::basic_format_args<basic_format_context>` - the underlying arguments are templates *on the context*.

We can't just... change the `basic_format_args` to have a different context, this is a fairly fundamental attachment in the design.

The *only* type for the output iterator that I can support in this implementation is precisely `fmt::appender`.

This seems like it'd be *extremely* limiting.

Except it turns out that actually nearly all of `libfmt` uses exactly this iterator. `fmt::print`, `fmt::format`, `fmt::format_to`, `fmt::format_to_n`, `fmt::vformat`, etc., all only use this one iterator type. This is because of [P2216R3]'s efforts to reduce code bloat by type erasing the output iterator.

However, there is one part of `libfmt` that uses a different iterator type, which the above implementation fails on:

```

fmt::format("{:::d}", vector{vector{'a'}, vector{'b', 'c'}}); // ok:
[[97], [98, 99]]
fmt::format(FMT_COMPILE("{:::d}"), vector{vector{'a'}, vector{'b', 'c'}}); // ill-
formed

```

The latter fails because there the initial output iterator type is `std::back_insert_iterator<std::string>`. This is a different iterator type from `fmt::appender`, so we get a mismatch in the types of the `basic_format_args` specializations, and cannot compile the construction of `bctx`.

This can be worked around (I just need to know what the type of the buffer needs to be, in the usual case it's `fmt::memory_buffer` and here it becomes `std::string`, that's fine), but it means we really need to nail down what the requirements of the formatter API are. One of the things we need to do in this paper is provide a formattable concept. From a previous revision of that paper, dropping the `char` parameter for simplicity, that looks like:

```

template <class T>
concept formattable-impl =

```

```

std::semiregular<fmt::formatter<T>> &&
requires (fmt::formatter<T> f,
          const T t,
          fmt::basic_format_context<char*, char> fc,
          fmt::basic_format_parse_context<char> pc)
{
    { f.parse(pc) } ->
    std::same_as<fmt::basic_format_parse_context<char>::iterator>;
    { f.format(t, fc) } -> std::same_as<char*>;
};

template <class T>
concept formattable = formattable_impl<std::remove_cvref_t<T>>;

```

I use `char*` as the output iterator, but my `range_formatter<V>` cannot support `char*` as an output iterator type at all. Do formatter specializations need to support any output iterator type? If so, how can we implement fill/align/pad support in `range_formatter`?

The simplest approach would be to state that there actually is only one output iterator type that need be support per character type. This is mostly already the case in `libfmt`, and seems to be how MSVC implements `<format>` as well. That is, we already have in 20.20.1 [\[format.syn\]](#):

```

namespace std {
    // [format.context], class template basic_format_context
    template<class Out, class charT> class basic_format_context;
    using format_context = basic_format_context<unspecified, char>;
    using wformat_context = basic_format_context<unspecified, wchar_t>;
}

```

The suggestion would be that the only contexts that need be supported are `std::format_context` and/or `std::wformat_context`. Only one context for each character type.

That reduces the problem quite a bit, but it's still not enough. We're not exposing what the buffer type needs to be, so even if I knew I only had to deal with `std::format_context`, I still wouldn't know how to construct a dynamic buffer that `std::format_context::iterator` is an extending output iterator into. That is, we need to expose/standardize `fmt::memory_buffer` (or provide it as an typedef somewhere).

If we don't require *just* one format context per character type, we can simply throw more type erasure at the problem. Say the only allowed iterators are either (using `libfmt`'s names) `fmt::appender` or `variant<fmt::appender, Out>`. The latter still allows support for other iterator types, while still letting other formatters use `fmt::appender` which they know how to do. This has some cost of course, but it does provide extra flexibility.

At a minimum, the API we need is:

```

template <typename V, typename FormatContext>
constexpr auto format(V&& value, FormatContext& ctx) -> typename
    FormatContext::iterator
{
    // ctx here is a basic_format_context<OutIt, CharT>, for some output iterator
    // and some character type

    // can use a vector<CharT>, basic_string<CharT>, or some custom buffer like
    // fmt::buffer, user's choice
    vector<CharT> buf;
}

```

```

// The retargeted_format_context class template can keep extra state if
// necessary, but bctx is still definitely a (w)format_context. The library
// ensures that regardless of the provided iterator, it gets type-erased as
// necessary
retargeted_format_context rctx(ctx, std::back_inserter(buf));
auto& bctx = rctx.context();

// format into bctx...
}

```

This can be made to work by `retargeted_format_context` simply doing the type erasure itself, and providing the user with the type-erased iterator result. Same as the library is already doing for all of its other entry points. For the typical case where all the entry points are already this type-erased iterator type, this is trivial. And if we allow arbitrary iterator types in the future, that entry point will have to erase both ways. Which is work, but it seems both quite feasible and in line with the rest of the design.

This could theoretically have been an ABI break, except that everything in the standard library today uses the one type-erased iterator (in which case the issue here is not a problem, except insofar as there is no way to actually create a new `format_context`).

§ 3.3.2 Manipulating `basic_format_parse_context` to search for sentinels

Take a look at one of the pair formatting examples:

```

fmt::format("{:|#x|^10}", std::pair(42, "hello"s));

```

In order for this to work, the `formatter<int>` object needs to be passed a context that just contains the string `"#x"` and the `formatter<string>` object needs to be passed a context that just contains the string `"^10"` (or possibly `"^10}"`). This is because `formatter<T>::parse` must consume the whole context. That's the API.

But `basic_format_parse_context` does not provide a way for you to take a slice of it, and we can't just construct a new object because of the dynamic argument counting support. Not just *any* context, but *specifically that one*.

Tim's suggested design for how to even do specifiers for pair also came with a suggested implementation: use a sentry-like type that temporarily modifies the context and restores it later. The use of this type looks like this:

```

auto const delim = *begin++;
ctx.advance_to(begin);
tuple_for_each_index(underlying, [&](auto I, auto& f){
    auto next_delim = std::find(ctx.begin(), end, delim);
    if constexpr (I + 1 < sizeof...(Ts)) {
        if (next_delim == end) {
            throw fmt::format_error("ran out of specifiers");
        }
    }
    end_sentry _(ctx, next_delim);
    auto i = f.parse(ctx);
    if (i != next_delim && *i != '}') {
        throw fmt::format_error("this is broken");
    }

    if (next_delim != end) {
        ++i;
    }
}

```



```
    ctx.advance_to(i);  
});
```

This ensures that each element of the pair/tuple only sees its part of the whole parse string, which is the only part that it knows what to do anything with.

Without something like this in the library, it'd be impossible to do this sort of complex specifier parsing. You could support ranges (there, we only have one underlying element, so it parses to the end), but not pair or tuple. We *could* say that since pair and tuple are library types, the library should just Make This Work, but there are surely other examples of wanting to do this sort of thing and it doesn't feel right to not allow users to do it too.

This design space is, thankfully, slightly easier than the previous problem: this is basically what you have to do. Not much choice, I don't think.

§ 3.3.3 Parsing of alignment, padding, and width

The first two issues in this section are serious implementation issues that require design changes to `<format>`. This one doesn't *require* changes, and this paper won't propose changes, but it's worth pointing out nevertheless. Alignment, padding, and width are the most common and fairly universal specifiers. But we don't provide a public API to actually parse them.

When implementing this in `fmt`, I just took advantage of `fmt`'s implementation details to make this a lot easier for myself: a type (`dynamic_format_specs<char>`) that holds all the specifier results, a function that understands those to let you write a padded/aligned string (`write`), and several parsing functions that are well designed to do the right thing if you have a unique set of specifiers you wish to parse (the appropriately-named `parse_align` and `parse_width`).

These don't have to be standardized, as nothing in these functions is something that a user couldn't write on their own. And this paper is big enough already, so it, again, won't propose anything in this space. But it's worth considering for the future.

§ 3.4 How to support those views which are not `const`-iterable?

In a previous revision of this paper, this was a real problem since at the time `std::format` accepted its arguments by `const Args&...`

However, [P2418R2] was speedily adopted specifically to address this issue, and now `std::format` accepts its arguments by `Args&&...`. This allows those views which are not `const`-iterable to be mutably passed into `format()` and `print()` and then mutably into its formatter. To support both `const` and non-`const` formatting of ranges without too much boilerplate, we can do it this way:

```
template <formattable V>  
struct range_formatter {  
    template <typename ParseContext>  
    constexpr auto parse(ParseContext&);  
  
    template <range R, typename FormatContext>  
        requires same_as<remove_cvref_t<range_reference_t<R>>, V>  
    constexpr auto format(R&&, FormatContext&);  
};  
  
template <range R> requires formattable<range_reference_t<R>>
```

```
struct formatter<R> : range_formatter<range_reference_t<R>>
{ };
```

`range_formatter` allows reducing unnecessary template instantiations. Any range of `int` is going to parse its specifiers the same way, there's no need to re-instantiate that code *n* times. Such a type will also help users to write their own formatters.

§ 3.5 What additional functionality?

There's three layers of potential functionality:

1. Top-level printing of ranges: this is `fmt::print("{} ", r)`;
2. A format-joiner which allows providing a custom delimiter: this is `fmt::print("{:02x}", fmt::join(r, " "))`. This revision of the paper allows providing a format specifier and removed in the brackets in the top-level case too, as in `fmt::print("{:e:02x}", r)`, but does not allow for providing a custom delimiter.
3. A more involved version of a format-joiner which takes a delimiter and a callback that gets invoked on each element. `fmt` does not provide such a mechanism, though the Rust `itertools` library does:

```
let matrix = [[1., 2., 3.],
              [4., 5., 6.]];
let matrix_formatter = matrix.iter().format_with("\n", |row, f| {
    f(&row.iter().format_with(" ", |elt, g| g(&elt)))
});
assert_eq!(format!("{}", matrix_formatter),
           "1, 2, 3\n4, 5, 6");
```

This paper suggests the first two and encourages research into the third.

§ 3.6 `format` or `std::cout`?

Just `format` is sufficient.

§ 3.7 What about `vector<bool>`?

Nobody expected this section.

The `value_type` of this range is `bool`, which is formattable. But the reference type of this range is `vector<bool>::reference`, which is not. In order to make the whole type formattable, we can either make `vector<bool>::reference` formattable (and thus, in general, a range is formattable if its reference types is formattable) or allow formatting to fall back to constructing a `value_type` for each reference (and thus, in general, a range is formattable if either its reference type or its `value_type` is formattable).

For most ranges, the `value_type` is `remove_cvref_t<reference>`, so there's no distinction here between the two options. And even for `zip` [P2321R2], there's still not much distinction since it just wraps this question in tuple since again for most ranges the types will be something like `tuple<T, U>` vs `tuple<T&, U const&>`, so again there isn't much distinction.

`vector<bool>` is one of the very few ranges in which the two types are truly quite different. So it doesn't offer much in the way of a good example here, since `bool` is cheaply constructible from `vector<bool>::reference`.

Though it's also very cheap to provide a formatter specialization for `vector<bool>::reference`.

Rather than having the library provide a default fallback that lifts all the reference types to `value_types`, which may be arbitrarily expensive for unknown ranges, this paper proposes a format specialization for `vector<bool>::reference`. Or, rather, since it's actually defined as `vector<bool, Alloc>::reference`, this isn't necessarily feasible, so instead this paper proposes a specialization for `vector<bool, Alloc>` at top level.

§ 4 Proposal

The standard library will provide the following utilities:

- A `formattable` concept.
- A `range_formatter<V>` that uses a `formatter<V>` to parse and format a range whose reference is similar to `V`. This can accept a specifier on the range (align/pad/width as well as string/map/debug/empty) and on the underlying element (which will be applied to every element in the range).
- A `tuple_formatter<Ts...>` that uses a `formatter<T>` for each `T` in `Ts...` to parse and format either a pair, tuple, or array with appropriate elements. This can accepted a specifier on the tuple-like (align/pad/width) as well as a specifier for each underlying element (with a custom delimiter).
- A `retargeted_format_context` facility that allows the user to construct a new `(w)` `format_context` with a custom output iterator.
- An `end_sentry` facility that allows the user to manipulate the parse context's range, for generic parsing purposes.

The standard library should add specializations of `formatter` for:

- any type `R` that is a range whose reference is formattable, which inherits from `range_formatter<remove_cvref_t<ranges::range_reference_t<R>>>`
- `pair<T, U>` if `T` and `U` are formattable, which inherits from `tuple_formatter<remove_cvref_t<T>, remove_cvref_t<U>>`
- `tuple<Ts...>` if all of `Ts...` are formattable, which inherits from `tuple_formatter<remove_cvref_t<Ts>...>`

Additionally, the standard library should provide the following more specific specializations of `formatter`:

- `vector<bool, Alloc>` (which formats as a range of `bool`)
- all the associative maps (`map`, `multimap`, `unordered_map`, `unordered_multimap`) if their respective key/value types are formattable. This accepts the same set of specifiers as any other range, except by *default* it will format as `{k: v, k: v}` instead of `[(k, v), (k, v)]`
- all the associative sets (`sets`, `multiset`, `unordered_set`, `unordered_multiset`) if their respective key/value types are formattable. This accepts the same set of specifiers as any other range, except by *default* it will format as `{v1, v2}` instead of `[v1, v2]`

Formatting for `string`, `string_view`, and `char/wchar_t` will gain a `?` specifier, which causes these types to be printed as escaped and quoted if provided. Ranges and tuples will, by default, print their elements as escaped and quoted, unless the user provides a specifier for the element.

The standard library should also add a utility `std::format_join` (or any other suitable name, knowing that `std::views::join` already exists), following in the footsteps of `fmt::join`, which allows the user to provide more customization in how ranges and tuples get formatted. Even though this paper allows you to provide a specifier for each element in the range, it does not let you change the delimiter in the specifier (that's... a bit much), so `fmt::join` is still a useful and necessary facility for that.

§ 4.1 Wording

None yet, since spent all my time on implementation but nevertheless wanted to get this paper out sooner.

§ 5 References

[LWG3478] Barry Revzin. `views::split` drops trailing empty range.

<https://wg21.link/lwg3478>

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.

<https://wg21.link/p2214r0>

[P2216R3] Victor Zverovich. 2021-02-15. `std::format` improvements.

<https://wg21.link/p2216r3>

[P2286R0] Barry Revzin. 2021-01-15. Formatting Ranges.

<https://wg21.link/p2286r0>

[P2286R1] Barry Revzin. 2021-02-19. Formatting Ranges.

<https://wg21.link/p2286r1>

[P2286R2] Barry Revzin. Formatting Ranges.

<https://wg21.link/p2286r2>

[P2321R2] Tim Song. 2021-06-11. `zip`.

<https://wg21.link/p2321r2>

[P2418R0] Victor Zverovich. Add support for `std::generator`-like types to `std::format`.

<https://wg21.link/p2418r0>

[P2418R2] Victor Zverovich. 2021-09-24. Add support for `std::generator`-like types to `std::format`.

<https://wg21.link/p2418r2>